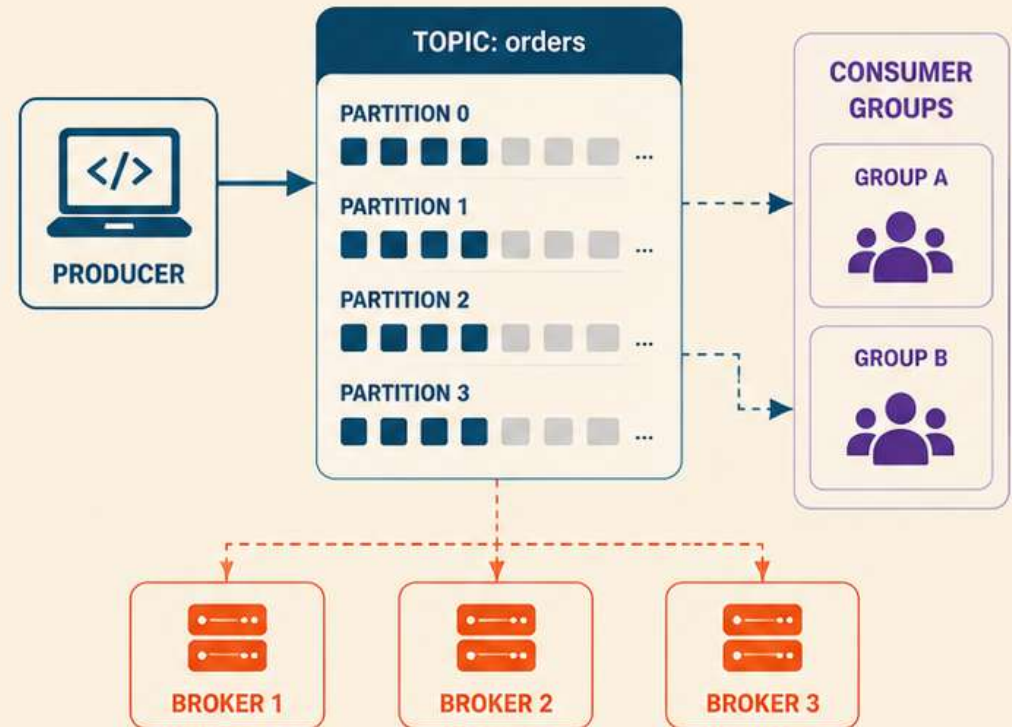


7 YOU ABSOLUTELY DON'T KNOW

KAFKA

IF YOU DON'T KNOW THESE 7 THINGS ABOUT KAFKA



 The must-know concepts that make Kafka powerful, scalable and reliable.



1. TOPIC IS A DISTRIBUTED LOG



2. PARTITIONS UNLOCK SCALABILITY



3. MESSAGE KEY CONTROLS MORE THAN YOU THINK



4. CONSUMER GROUPS ENABLE SCALABLE PROCESSING



5. OFFSETS TRACK YOUR POSITION



6. REPLICATION MAKES KAFKA FAULT-TOLERANT



7. KAFKA USES KRAFT, NOT ZOOKEEPER



**MASTER THESE 7 CONCEPTS.
BUILD SYSTEMS THAT SCALE.**



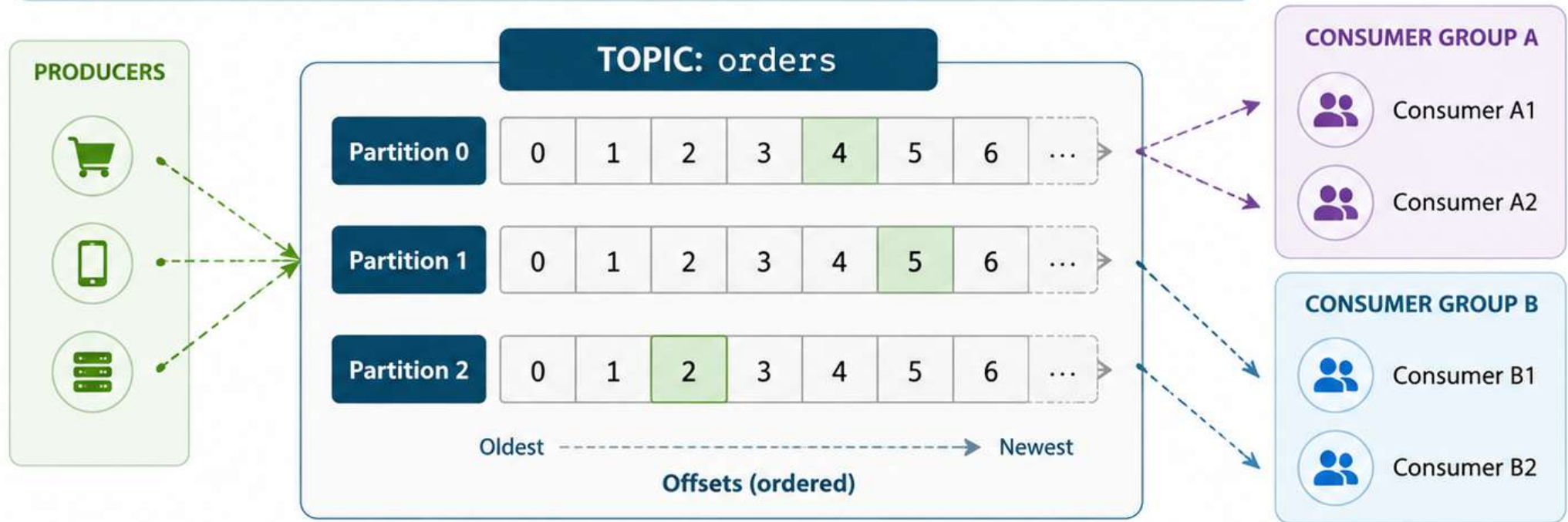
FOLLOW ON LINKEDIN
for more insights on Kafka,
Streaming & Real-Time Systems

1 A Topic is not a Queue — It's a **Distributed Log**



Kafka doesn't delete a message simply because a consumer has read it.

Records remain available according to the retention policy, which means multiple consumer groups can independently consume the same event stream.



TOPIC

A logical category or feed name where events are published.



PARTITION

A unit of parallelism. Each partition is an ordered, append-only log of records.



OFFSET

A unique sequential number for each record in a partition.



RETENTION

Controls how long records are kept before being deleted.



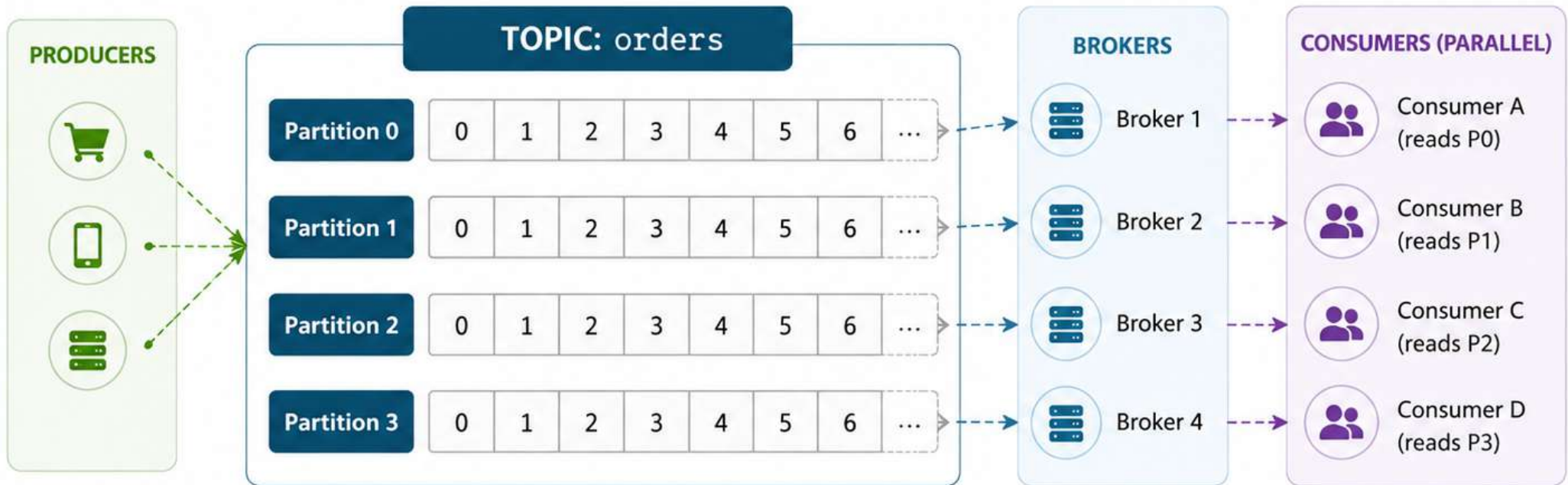
2

Partitions Are the Secret Behind Kafka's Scalability



A topic isn't processed as one giant stream.

It's split into partitions, which spread data across brokers and enable parallel processing by consumers.



WHY PARTITIONS MATTER

- ✓ Distributes data across brokers for scalability
- ✓ Enables parallel processing by multiple consumers
- ✓ Improves throughput and fault tolerance



CRUCIAL LIMITATION

Kafka guarantees ordering within a partition, not across the entire topic.

Messages from different partitions can be processed out of order.

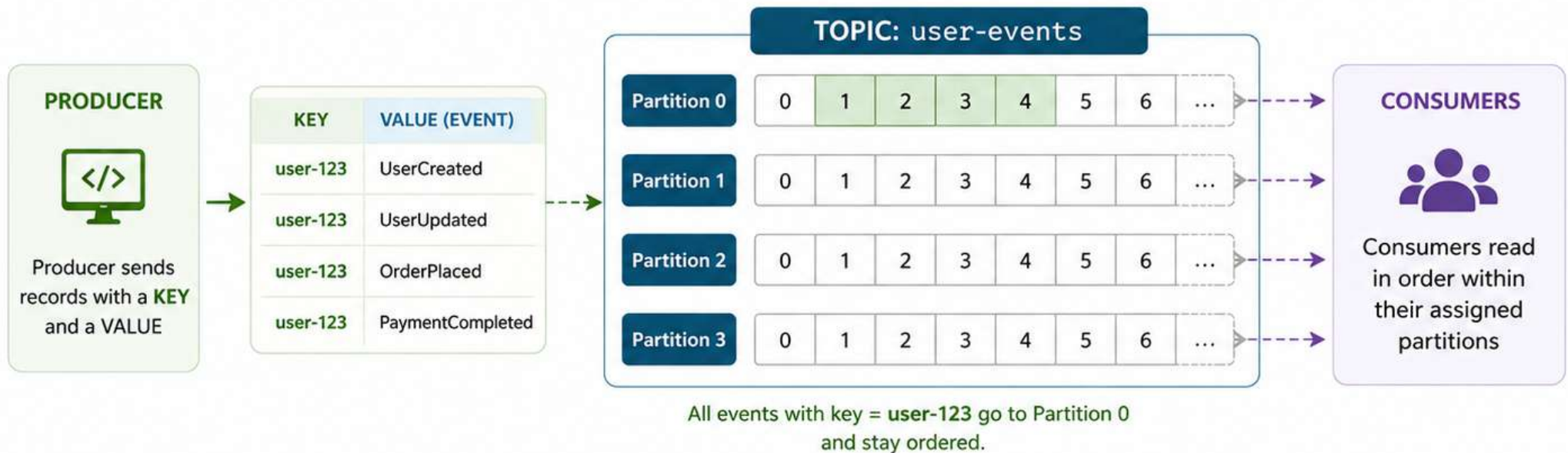


3 The Message Key Controls More Than You Think



The key determines which partition a record is written to.

Using the same key for related events sends them to the same partition, preserving order within that partition.



HOW IT WORKS

Kafka uses a hashing algorithm on the key to choose a partition.

- Same key → Same partition
- Different key → Can go to different partitions



EXAMPLE

key = user-123

→ All events for user-123 land in the same partition and remain ordered.

key = user-456

→ May go to a different partition.



KEY TAKEAWAY

The key isn't just metadata.

Your key-selection strategy can determine your ordering guarantees and workload distribution.



4

Consumer Groups Turn Kafka Into a Scalable Processing System



Each partition is consumed by one consumer within a group.

Kafka balances partitions across consumers in the same group, enabling parallel processing and horizontal scalability.

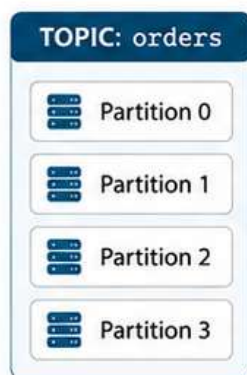


IMPORTANT LIMITATION

Adding consumers beyond the number of partitions doesn't increase parallelism for that topic.

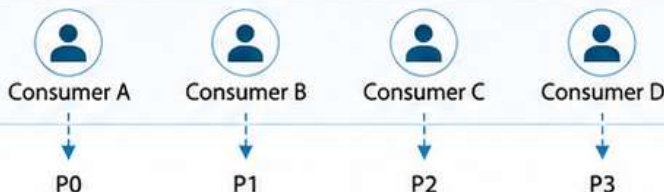
KEY TAKEAWAY

Scale by increasing partitions or by creating more consumer groups for the same topic.



EXAMPLE 1: 4 PARTITIONS, 4 CONSUMERS IN A GROUP

Consumer Group: order-processors



One consumer per partition.
Perfect parallelism.

EXAMPLE 2: ADD ANOTHER CONSUMER (5 CONSUMERS)

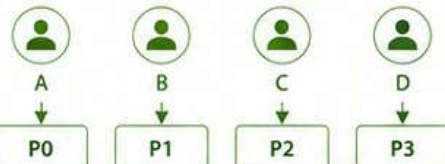
Consumer Group: order-processors



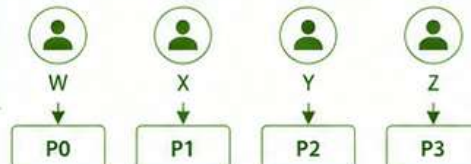
No increase in parallelism.
Extra consumer will be idle.

EXAMPLE 3: SAME TOPIC, DIFFERENT CONSUMER GROUPS

Consumer Group: order-processors



Consumer Group: analytics-app



Each group gets its own view and progress.
No duplication of data.



MULTIPLE CONSUMER GROUPS

Same topic, independent consumption.



HOW IT WORKS



Kafka assigns partitions to consumers in a group.



When consumers join or leave, Kafka rebalances partitions among them.



More partitions = more parallelism opportunity.

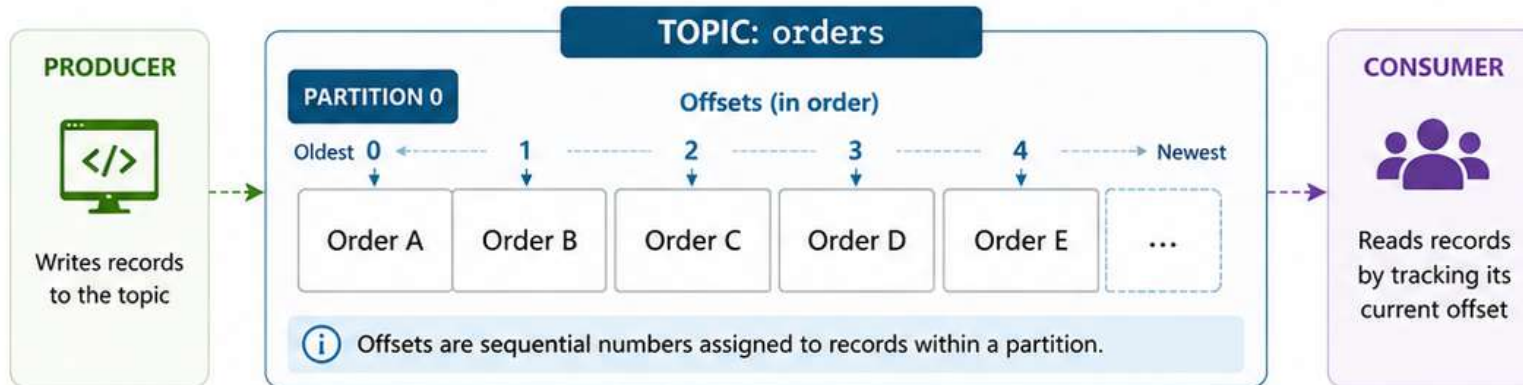


5 Offsets Are How Kafka Knows Where You Are



Each record in a partition has a unique offset.

Consumers track their position using offsets. Kafka retains the log; consumers decide what to read.



WHAT OFFSETS ENABLE



RESUME

Continue reading from where you left off.



REPLAY

Re-read historical records for reprocessing or debugging.



FLEXIBLE CONSUMPTION

Consume as per retention policy and business needs.



RELIABLE PROCESSING

Track, commit, and manage progress safely.

KAFKA IS NOT A TRADITIONAL QUEUE

TRADITIONAL QUEUE



Message in queue



Message consumed → removed

VS

KAFKA (DISTRIBUTED LOG)



Message appended to the log



Consumer tracks position (offset)



Log retained as per retention



Once consumed, the message is gone.



Messages remain; consumers track their position.

OFFSETS IN ACTION (EXAMPLE)



Consumer starts at offset 2

Partition 0



Consumer will read from offset 2 → 3 → 4 → 5 ...

If the consumer restarts, it can continue from the last **committed** offset (e.g., 4)



KEY TAKEAWAY

Offsets are the identity of each record's position in a partition. They are the reason Kafka can retain data, enable can replay, and support multiple independent consumers.



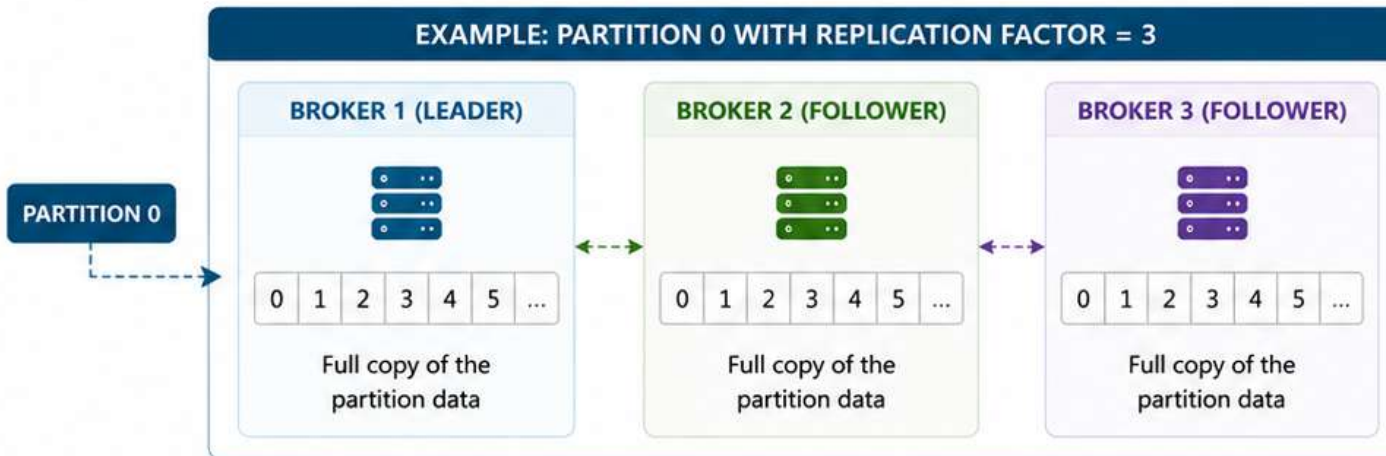
6

Replication Is What Makes Kafka Fault-Tolerant



Partitions have multiple replicas across brokers.

If the leader broker fails, Kafka automatically elects a new leader from the replicas to keep the system available and your data safe.



WHAT HAPPENS IF THE LEADER FAILS?

1. LEADER (Broker 1) FAILS



2. KAFKA ELECTS A NEW LEADER



KEY CONCEPTS



LEADER

Handles all read and write requests for the partition.



FOLLOWER (REPLICA)

Keeps an up-to-date copy of the leader's data.



REPLICATION FACTOR (RF)

Number of copies of each partition across brokers.



FAULT TOLERANCE

With replication factor N , Kafka can tolerate up to $N-1$ broker failures without losing committed data*.

* Subject to the relevant durability configuration.



No data loss.
No downtime.
The partition stays available.



WHY REPLICATION MATTERS

- ✓ Protects against broker crashes
- ✓ Ensures high availability
- ✓ Keeps your data safe and your system running



GOOD TO KNOW

Higher replication factor = more fault tolerance, but uses more storage and increases cost.
Common choice: RF = 3 for production.



7

Kafka No Longer Needs ZooKeeper — Meet KRaft



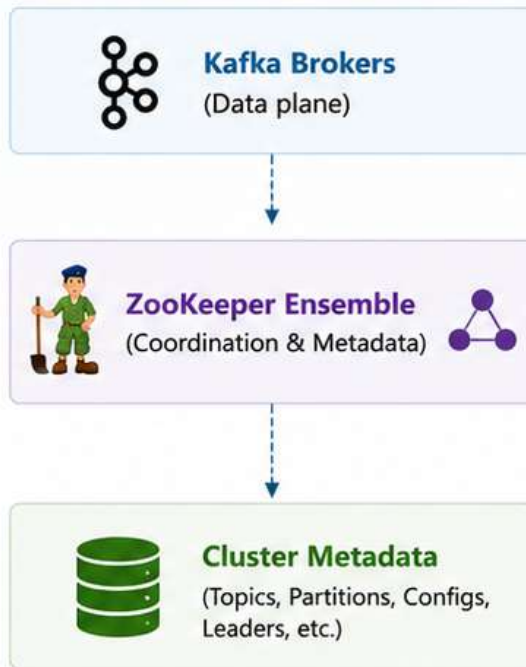
Modern Kafka uses KRaft (Kafka Raft) instead of ZooKeeper.

KRaft is Kafka's built-in metadata system using Raft consensus to provide high availability, consistency, and simpler operations.



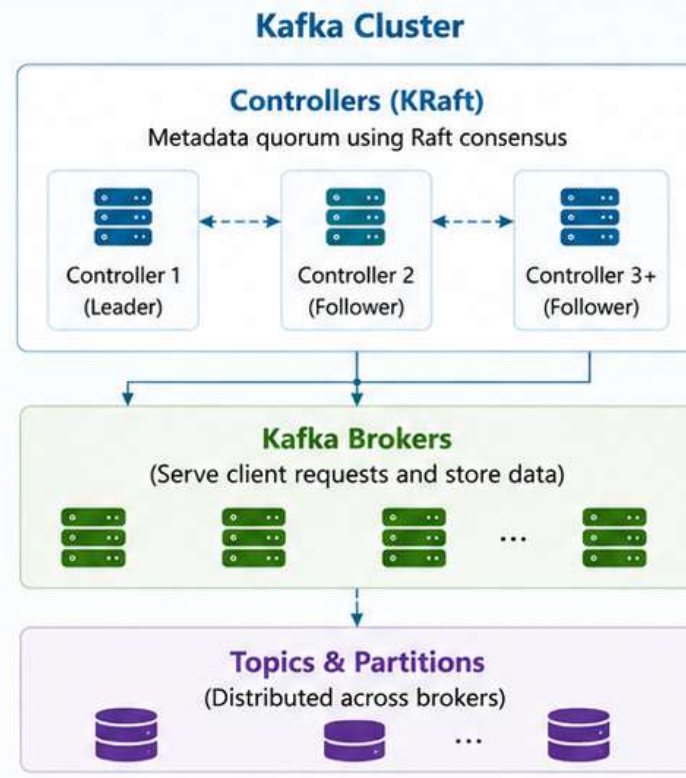
ZooKeeper is no longer required for new Kafka deployments (Kafka 3.3+ and Kafka 4.x).

OLD ARCHITECTURE: WITH ZOOKEEPER



Evolution
of Kafka
architecture

NEW ARCHITECTURE: WITH KRAFT



WHY KRAFT?



SIMPLER OPERATIONS

No separate ZooKeeper cluster to deploy, monitor, and maintain.



BETTER PERFORMANCE

Lower latency for metadata operations.



STRONGER CONSISTENCY

Built on Raft consensus for reliable metadata management.



BUILT FOR THE FUTURE

Default architecture in Kafka 4.x and beyond.



Controller nodes and broker nodes are separate roles.

Recommended: 3 or more controllers for high availability.



KEY TAKEAWAY

KRaft eliminates the dependency on ZooKeeper by using Kafka's own Raft-based controller quorum to manage cluster metadata. It's simpler, faster, and the future of Kafka.





YOU'VE LEARNED THE 7 THINGS

NOW, PUT THIS KNOWLEDGE
INTO ACTION.

KEEP EXPLORING. KEEP BUILDING.



FOLLOW FOR MORE INSIGHTS
ON KAFKA, STREAMING & REAL-TIME SYSTEMS



LET'S BUILD DATA-DRIVEN FUTURES TOGETHER.

